

Temario A

Se desea realizar una aplicación que posibilite automatizar la asignación de muelles en las operaciones de carga y descarga en un puerto de nuestro país. Para que el sistema logre realizar la correcta asignación muelle – barco necesita tener en cuenta determinadas reglas, los barcos son atendidos de uno en uno y cada muelle tiene asignados no más de 5 barcos. En los muelles se atienden tres tipos de embarcaciones que pueden clasificarse en Trasatlánticos (barcos de pasajeros), Mercantes y Tanqueros (de carga). De los tipos de barcos el sistema necesita controlar los siguientes datos:

Trasatlánticos: Nombre del barco, país de procedencia, cantidad de camarotes con que cuenta.

Mercantes: Capacidad total de carga (toneladas), nombre del barco, cantidad de grúas de descarga con que cuenta, país de procedencia.

Tanqueros: Volumen total de carga (m3), cantidad de bombas de extracción que cuenta, país de procedencia y nombre del barco

Independientemente del tipo de un barco también se controla su año de fabricación y la dimensión del mismo.

Un puerto está formado por un conjunto de muelles de los cuales el sistema controla: número, dimensión, tipo de barco que atiende (de pasajero o mercancía), si es para carga o para descarga y cantidad de obreros que laboran en él.

Tanto para la operación de descarga como para la de carga de un barco se necesita de una cierta cantidad mínima de hombres laborando en el muelle. Esta cantidad se determina teniendo en cuenta el tipo de barco y las características del mismo, la manera en que se determina esta cantidad queda recogida en la siguiente tabla:

Tipo de Barco	Cantidad Mínima de Hombres
Trasatlántico	Total de camarotes / 5
Mercante	cantidad de grúas * (Capacidad total de carga - 100 * cantidad de grúas)
Tanquero	cantidad de bombas * (Volumen total de carga - 1000 * cantidad de bombas)

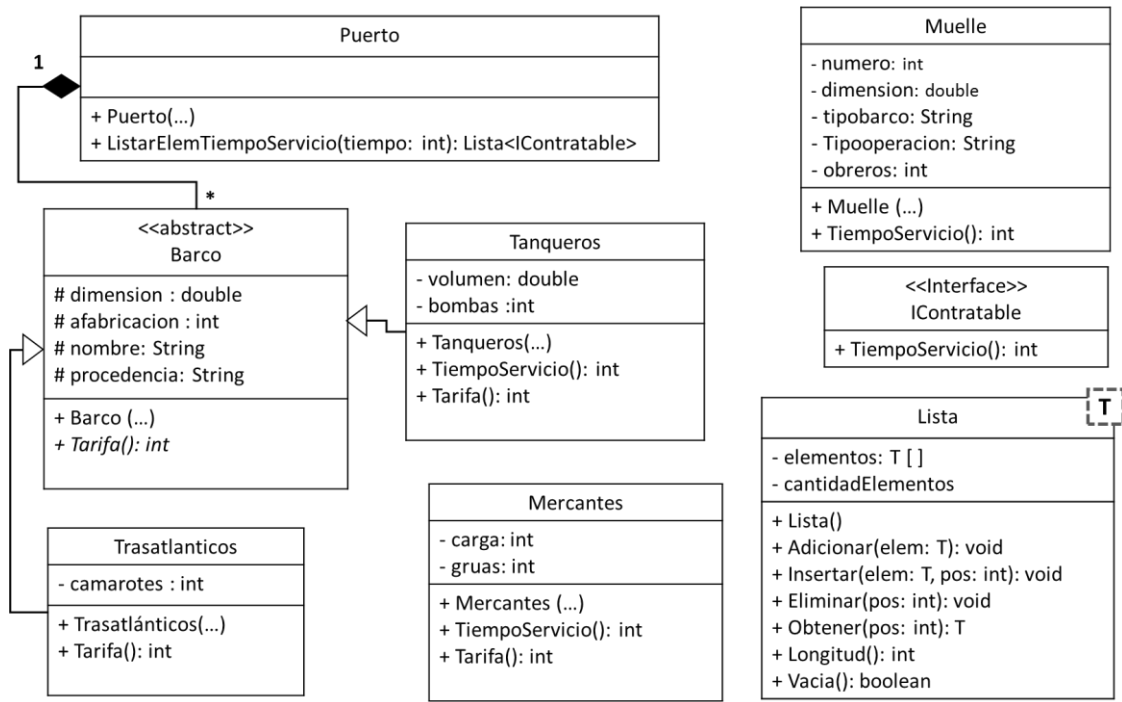
Nota: Todos estos valores se redondean por exceso, es decir, al valor entero superior.

El puerto tiene una tarifa por el servicio prestado, sea carga o descarga y se determina de la siguiente forma:

Tipo de Barco	Tarifa
Trasatlántico	50 * Cantidad Mínima de Hombres
Mercante	100 * Cantidad Mínima de Hombres
Tanquero	150 * Cantidad Mínima de Hombres

Tanto los barcos de carga como los muelles son entidades que se rigen por normas internacionales para prestar servicios haciéndose necesario que se conozca su tiempo de servicio por lo que cuentan con una funcionalidad que les permite determinar dicho tiempo, ya que a partir de este se puede determinar si son contratables o no. El puerto por su parte, debe permitir obtener el listado de los elementos que poseen un determinado con tiempo de servicio.

Teniendo en cuenta la descripción anterior y el diagrama de clase que se modela a continuación, responda los siguientes incisos:



1. Complete el diagrama de clases dado con las relaciones y métodos que faltan, para que se ajuste al problema descrito.
2. Implemente, en las clases correspondientes, los métodos polimórficos ubicados por usted en el diagrama de clases.
3. Declare la clase controladora haciendo uso de la clase Lista<T> para la representación de los Barcos.
4. Implemente en la clase correspondiente:

- a. El método **BarcoMashombres():Barco**, el cual retorna barco que más hombres necesita para ser atendido.
- b. El método **TarifaPromedioTrasatlantico(cant: int):double**, el cual retorna la tarifa promedio por servicio prestado a los barcos trasatlánticos con una cantidad de camarotes dados.

Nota: Asuma que todos los métodos de acceso a los atributos ya están implementados.

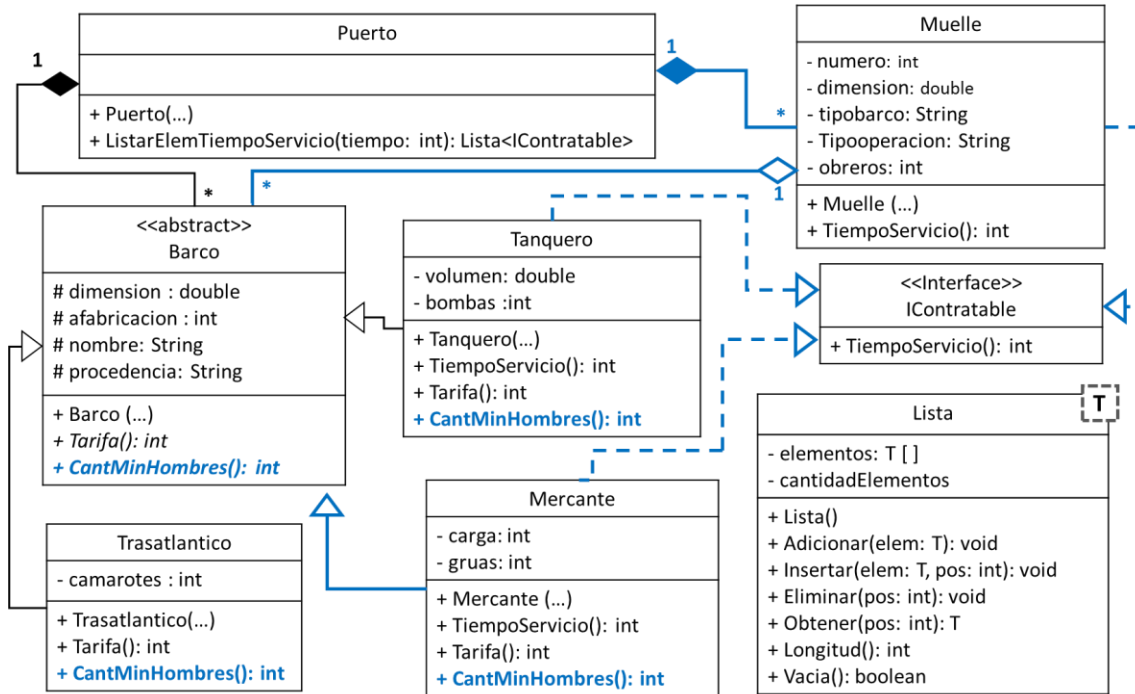
5. Declare una clase genérica nombrada ListaOrd y que herede de la clase Lista<T> haciéndola concreta para datos enteros. Implemente el método **OrdenadosAsc():Lista<Integer>** que retorna la lista ordenada de forma ascendente de los elementos de las posiciones impares. Lance una excepción si la colección esta vacia, para ello cree una nueva clase de excepciones llamada **ListaOrdVaciaException**.
6. Implemente en las clases correspondientes, los métodos necesarios para salvar en el fichero binario "mercantes.bin" y separados por un cambio de línea: el nombre, la capacidad total de carga y el tiempo de servicio de los barcos mercantes. Realice el manejo de excepciones correspondiente de modo que el fichero siempre se cierre, aun cuando se interrumpa la ejecución del método por ocurrir algún error en el proceso de escritura.

Clases para el trabajo con ficheros en Java

File	FileReader	FileWriter	BufferedReader	FileInputStream
BufferedWriter	FileOutputStream	DataInputStream	ObjectInputStream	ObjectOutputStream
DataOutputStream				

Respuestas del Temario A

1)



2) **Clase Barco**

```
public abstract int CantMinHombres();
```

Clase Trsatlantico

```
public int CantMinHombres(){ return camarotes / 5 ; }
```

Clase Mercante

```
public int CantMinHombres(){ return gruas * (carga - 100 * gras); }
```

Clase Tanquero

```
public int CantMinHombres(){ return bombas * (volumen - 1000 * bombas); }
```

3) public class Puerto {

```
    Lista<Barco> barcos;
```

```
    Lista<Muelle> muelles;
```

```
}
```

4) a) public Barco BarcoMashombres(){

```
    Barco b = barcos.Obtener(0);
```

```
    for(int i = 1; i < barcos.Longitud(); i++){
```

```
        if(barcos.Obtener(i).CantMinHombres() > b. CantMinHombres())
```

```
            b = barcos.Obtener(i);
```

```
    }
```

```
    return b;
```

```
}
```

b) public double TarifaPromedioTrsatlantico (int cant){

```
    int sum = 0, cont = 0;
```

```
    for(int i = 1; i < barcos.Longitud(); i++)
```

```
        if(barcos.Obtener(i) instanceof Trsatlanticos)
```

```
            if ( ((Trsatlanticos)barcos.Obtener(i)).getCamarotes() == cant)
```

```
                { sum+= barcos.Tarifa(); cont ++; }
```

```
    return sum/cont;
```

```
}
```

```

5) public class ListaOrd extends Lista<Integer>{
    public ListaOrd () { super(); }
    public Lista<Integer> OrdenadosAsc () throws ListaOrdVaciaException {
        if (Vacía()) throw new ListaOrdVaciaException();

        Lista<Integer> ord = new Lista<Integer> ();
        for(int i = 1; i < Longitud(); i+=2)
            ord.Adicionar(obtener(i));

        for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - i - 1; j++)
                if (ord.Obtener(j) > ord.Obtener(j + 1)) {
                    int aux = ord.Obtener(j);
                    ord.Eliminar(j);
                    ord.Insertar(aux,j+1);
                }
        return ord;
    }
}

public class ListaOrdVaciaException extends Exception{
    public ListaOrdVaciaException()
    {
        super("lista vacía");
    }
}

```

6) Clase Mercante

```

public void Salvar(DataOutputStream dos) throws IOException{
    dos.writeChars( Nombre + "\n" );
    dos.writeInt(carga);
    dos.writeInt( TiempoServicio() );
}

```

Clase Puerto

```

public void SalvarTodo () throws FileNotFoundException, IOException {
    FileOutputStream fos = new FileOutputStream("mercantes.bin");
    DataOutputStream dos = new DataOutputStream(fos);
    try {
        for (int i = 0; i < barcos.Longitud(); i++) {
            if(barcos.Obtener(i) instanceof Mercante){
                dos.writeChars("\n");
                ((Mercante) barcos.Obtener(i)).Salvar(dos);
            }
        }
    } finally { dos.close(); }
}

```

Temario B

En una biblioteca se desea automatizar el control de préstamos a los usuarios. Para atender las peticiones de los clientes, la biblioteca va almacenando en un listado las solicitudes de los mismos. De cada solicitud se recoge nombre, carné de identidad y dirección del cliente, así como si es asociado o no y una lista de elementos a solicitar (libro, revista, acta).

De cada uno de los materiales que se almacenan en la biblioteca se recogen: identificador, título, año de publicación, año de llegada a la biblioteca, editorial y cantidad de préstamos.

Los materiales se clasifican en libro, revista y actas de congreso, de los cuales se recogen además los siguientes datos:

Libro: Autor y género (Infantil, Ciencia ficción, historia antigua).

Revista: Cantidad de artículos y frecuencia publicación (trimestral, semestral, anual).

Actas de congreso: Nombre del congreso y cantidad de páginas.

Existe para cada material (libro, Revista, Actas) un **factor de estancia** que se calcula de la siguiente manera:

Tipo de Material	Factor de estancia
Libro	$(\text{AñoPublicado} + 1) / \text{AñoLlegaBiblioteca} * 1.05$
Revista	Trimestral: $(\text{MesesPublicada} + 1) / \text{AñoLlegaBiblioteca} * 1.4$ Anual: $(\text{MesesPublicada} + 1) / \text{AñoLlegaBiblioteca} * 1.25$
Actas	$(\text{AñoPublicado} + 1) / \text{AñoLlegaBiblioteca}$

Cada material tiene asociado un **índice de deterioro** debido al proceso de préstamos al que son objetivo, por experiencia se conoce que este índice varía según el tipo de material, este valor se calcula de la siguiente forma:

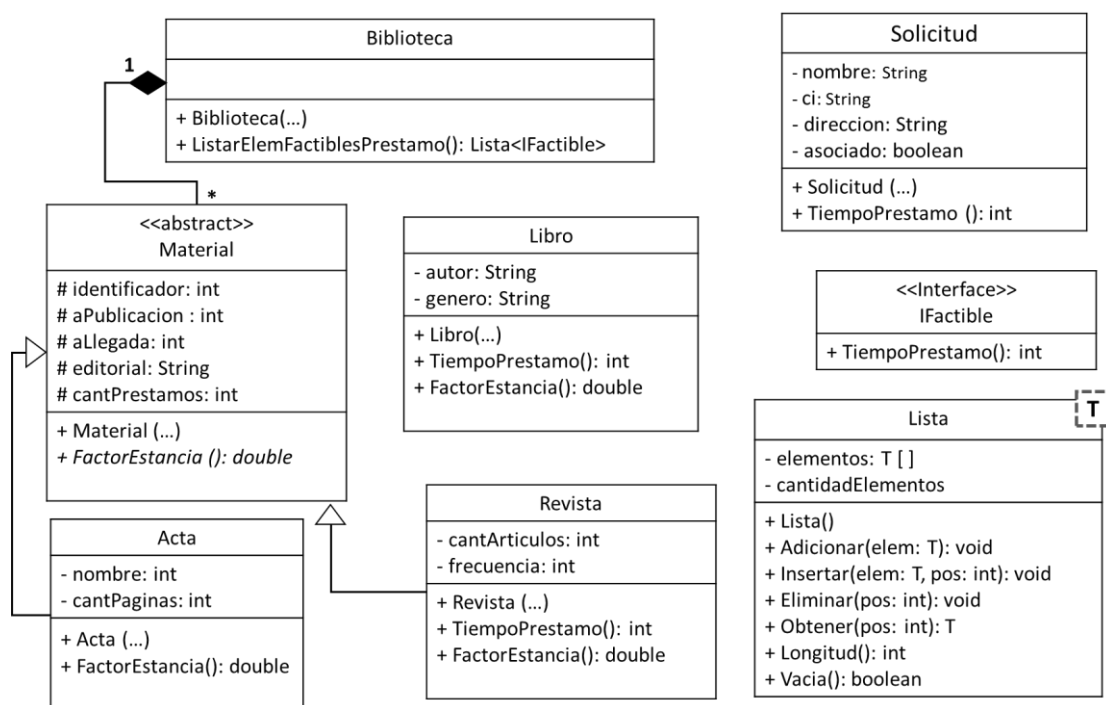
Tipo de Material	Índice de deterioro
Libro	$\text{Factor de estancia} * \text{cantidad de préstamos} / 0.6$
Revista	$\text{Cantidad de préstamos} / (\text{cantidad de artículos} * \text{Factor de estancia})$
Actas	$\text{Cantidad de páginas} / \text{cantidad de préstamos} * 0.5$

Los Libros y Revistas son los materiales más prestados y es interés de la biblioteca llevar un registro con el tiempo de préstamo de los mismos, de igual manera de las solicitudes es conocido el tiempo de préstamo. Es por ello que cuentan con una funcionalidad que les permite determinar dicho tiempo, ya que esto ayuda a determinar si son factibles los préstamos. La biblioteca por su parte, debe permitir obtener el listado de los elementos que son factibles para préstamos.

Teniendo en cuenta la descripción anterior y el diagrama de clase que se modela a continuación, responda los siguientes incisos:

Clases para el trabajo con ficheros en Java

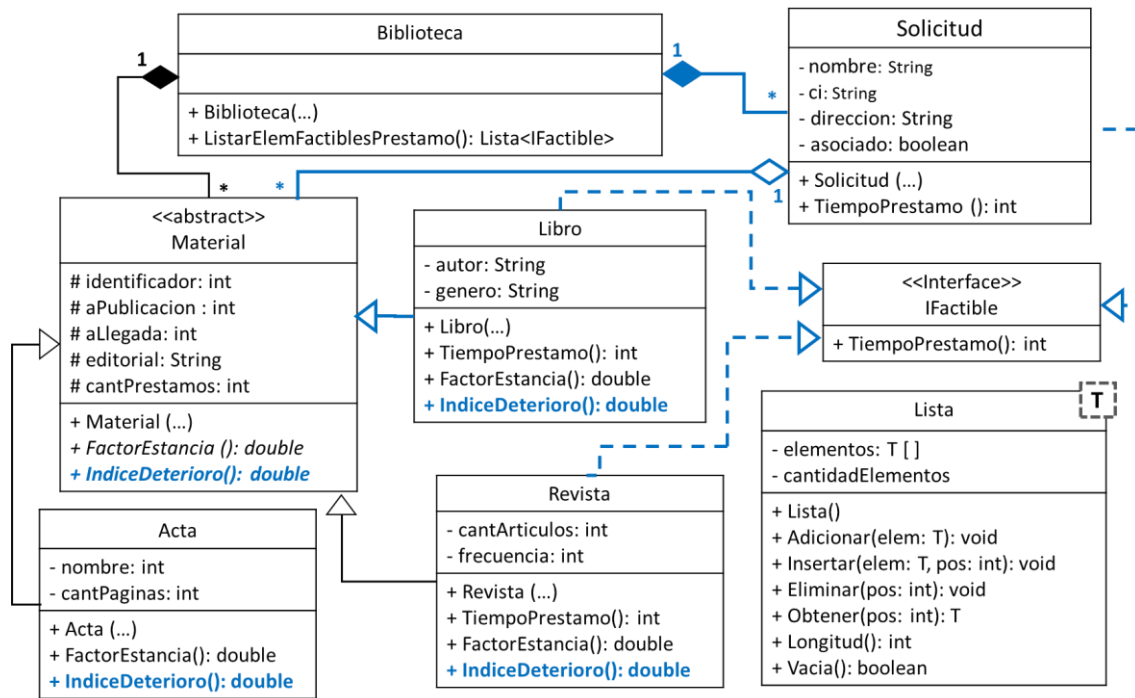
File	FileReader	FileWriter	BufferedReader	FileInputStream
BufferedWriter	FileOutputStream	DataInputStream	ObjectInputStream	ObjectOutputStream
DataOutputStream				



1. Complete el diagrama de clases dado con las relaciones y métodos que faltan, para que se ajuste al problema descrito.
2. Implemente, en las clases correspondientes, los métodos polimórficos ubicados por usted en el diagrama de clases.
3. Declare la clase controladora e implemente su constructor, haciendo uso de la clase `Lista<T>` para la representación de los Materiales.
4. Implemente en la clase correspondiente:
 - a. El método ***LibroMenosIndiceDeterioro():Libro***, el cual retorna Libro que menos índice de deterioro posee.
 - b. El método ***PorcientoFactorEstancia (valor:double):double***, el cual retorna el porciento que representan los materiales con un factor de estancia menor que un valor dado.
5. Declare una clase genérica nombrada *ListaPar* y que herede de la clase `Lista<T>` haciéndola concreta para datos enteros. Implemente el método ***ParesDesc():Lista<Integer>*** que retorna la lista ordenada de forma descendente de los elementos pares. Lance una excepción si la colección esta Vacía, para ello cree una nueva clase de excepciones llamada ***ListaParVacíaException***.
6. Implemente en las clases correspondientes, los métodos necesarios para salvar en el fichero de texto *"actas.txt"* y separados por un cambio de línea: la editorial, el nombre y el factor de estancia de las Actas. Realice el manejo de excepciones correspondiente de modo que el fichero siempre se cierre, aun cuando se interrumpa la ejecución del método por ocurrir algún error en el proceso de escritura.

Respuestas del Temario B

1)



2) Clase Material

```
public abstract double IndiceDeterioro();
```

Clase Libro

```
public double IndiceDeterioro () { return FactorEstancia()*cantPrestamos / 0.6 ; }
```

Clase Revista

```
public double IndiceDeterioro () { return cantPrestamos/ (cantArticulos* FactorEstancia()); }
```

Clase Acta

```
public double IndiceDeterioro () { return cantPaginas / cantPrestamos * 0.5; }
```

3) public class Biblioteca {

```
    Lista<Material> materiales;
```

```
    Lista<Solicitud> solicitudes;
```

```
}
```

4) a) public Libro LibroMenosIndiceDeterioro () {

```
    Libro lib = null; indDet = Double.MaxValue;
```

```
    for(int i = 1; i < materiales.Longitud(); i++)
```

```
        if(materiales.Obtener(i) instanceof Libro && materiales.Obtener(i).IndiceDeterioro() < indDet)
            { lib = materiales.Obtener(i); indDet = materiales.Obtener(i).IndiceDeterioro(); }
```

```
    return lib;
```

```
}
```

b) public double PorcentajeFactorEstancia (double valor){

```
    int cont = 0,
```

```
    for(int i = 0; i < materiales.Longitud(); i++)
```

```
        if (materiales.Obtener(i).FactorEstancia () < valor)
```

```
            { cont ++; }
```

```
    return cont /materiales.Longitud()*100; }
```

```

5) public class ListaPar extends Lista<Integer>{
    public ListaPar () { super(); }

    public Lista<Integer> ParesDesc() throws ListaParVaciationException {
        if (Vacía()) throw new ListaParVaciationException ();

        Lista<Integer> par = new Lista<Integer> ();
        for(int i = 0; i < Longitud(); i++)
            if(Obtener(i)%2==0) par.Adicionar(Obtener(i));

        for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - i - 1; j++)
                if (par.Obtener(j) < par.Obtener(j + 1)) {
                    int aux = par.Obtener(j);
                    par.Eliminar(j);
                    par.Insertar(aux,j+1);
                }
    }

    public class ListaParVaciationException extends Exception{
        public ListaParVaciationException () { super("lista vacía"); }
    }
}

```

6) Clase Acta

```

public void Salvar(BufferedWriter bw) throws IOException {
    bw.write(editorial + "\n");
    bw.write(nombre + "\n");
    bw.write(FactorEstancia()+ "\n");
}

```

Clase Puerto

```

public void SalvarTodo () throws FileNotFoundException, IOException {
    FileWriter fw = new FileWriter(fi);
    BufferedWriter bw = new BufferedWriter(fw);
    try {
        for (int i = 0; i < materiales.Longitud(); i++) {
            if(materiales.Obtener(i) instanceof Acta){
                bw.newLine();
                ((Acta) barcos.Obtener(i)).Salvar(dos);
            }
        }
    } finally { dos.close(); }
}

```